Software Requirements Specification

for

Project Vhagar: A Behavioral Finance Intelligence Platform

Version 1.0 approved

Prepared by Parth Kurve, Vishal Raut, Vivek Ukande

Ramdeobaba University, Nagpur

21/10/2026

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ..... **ii**
Revision History ..... **ii**
1. Introduction **1**
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description **2**
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements **3**
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features **4**
4.1 Evidence Registration 4
4.2 Custody Transfer (Secure Handshake Mechanism) 4
4.3 Immutable Ledger Integrity Mechanism 4
4.4 Audit Mode (Physical Inventory Verification) 4
4.5 User and Role Management 4
5. Other Nonfunctional Requirements **4**
5.1 Performance Requirements 4
5.2 Security Requirements 5
5.3 Reliability Requirements 5
5.4 Availability Requirements 5
5.5 Usability Requirements 5
5.6 Maintainability Requirements 5
5.7 Portability Requirements 5
5.8 Safety Requirements 5
5.9 Legal and Compliance Requirements 5
6. Other Requirements **6**
6.1 Glossary 6
6.2 System Architecture Model 6
6.3 Database Schema Overview 6
6.4 Future Enhancements 6
6.5 References 6

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a comprehensive, rigorous definition of Project Vhagar: A Behavioral Finance Intelligence Platform. This document outlines the complete set of functional and non-functional requirements governing the system's architecture, computational engines, user interfaces, and operational constraints.

In contemporary retail trading environments, individual investors routinely suffer severe financial losses due to unmitigated cognitive biases and irrational decision-making frameworks. According to the Securities and Exchange Board of India (SEBI) report published in August 2026, 87.7% of individual retail traders in the equity derivatives segment incurred net losses. Traditional trading platforms provide technical indicators and transactional mechanics but fail to monitor or intervene in behavioral anomalies such as loss-chasing, overtrading, or confirmation bias.

Project Vhagar addresses these critical systemic vulnerabilities by introducing real-time behavioral analytics, cognitive bias detection algorithms, and automated trade interruption mechanisms. This SRS serves as the authoritative baseline for system design, implementation, and empirical validation.

This document serves the following purposes:

a) To define system functionality in precise and measurable terms.
b) To establish technical constraints and design boundaries.
c) To provide a reference for implementation and system validation.
d) To assist evaluators in assessing system completeness and feasibility.
e) To support future maintenance and enhancement planning.

This specification applies to Version 1.0 of Project Vhagar, detailing requirements across the React.js frontend dashboard, Python/FastAPI backend analytical microservices, PostgreSQL relational database management system, cognitive evaluation engines, and automated execution control protocols.

1.2 Document Conventions

This document follows the IEEE 830-1998 Software Requirements Specification structure to ensure clarity, traceability, and standardization. The hierarchical numbering format enables easy reference and cross-linking between sections.

The following conventions are used throughout this document:

a) "Shall" indicates mandatory requirements that must be satisfied.
b) "Should" indicates recommended features that improve usability or performance.
c) "May" indicates optional features that can be considered in future versions.

All requirements are written to be:

• Specific
• Measurable
• Achievable
• Relevant
• Testable

Technical terminology relating to behavioral economics, quantitative finance, RESTful API architecture, JWT authentication, and trade execution latency is applied in accordance with established academic and IEEE standards.

1.3 Intended Audience and Reading Suggestions

This document is intended for multiple stakeholders involved in the system lifecycle.

The primary audience includes:

a) Quantitative Software Engineers responsible for backend analytics and API development.
b) Frontend Developers implementing user interface components and real-time visualization.
c) Quality Assurance and Verification Engineers validating requirements compliance.
d) System Administrators overseeing infrastructure deployment and security enforcement.

Developers should review Sections 3 and 4 carefully before beginning implementation, as these sections describe external interfaces and detailed functional workflows. Testing engineers should derive test cases from Sections 4 and 5 to ensure requirement coverage. Administrators should review Sections 2 and 5 to understand environmental constraints and security policies.

Readers unfamiliar with the project should begin with Section 2 to gain an understanding of system architecture, operating context, and user classifications before examining detailed specifications.

1.4 Product Scope

Project Vhagar is an advanced behavioral finance intelligence platform engineered to identify, quantify, and mitigate cognitive biases in real-time retail trading operations. The platform operates as an analytical and intervention layer integrated with trading execution environments.

The system achieves this through the integration of the following core mechanisms:

a) Real-time trade telemetric logging and behavioral pattern profiling.
b) Automated detection of cognitive biases (e.g., loss-aversion, overconfidence, overtrading).
c) Dynamic trade interruption protocols to enforce cooling-off periods during emotional distress.
d) Behavioral risk scoring and performance analytics generation.
e) Role-based administrative and configuration dashboards.

The system utilizes a high-throughput backend architecture powered by Python and FastAPI, supported by PostgreSQL for persistent relational data storage and analytical state management.

The platform does not act as an independent brokerage or custodian of capital; rather, it functions as an intelligence and behavioral safeguarding module operating alongside market trading interfaces.

The primary objective of the system is to provide accountability, transparency, and cryptographic integrity in evidence handling workflows.

1.1 References

The development and documentation of this system are based on established technical standards and authoritative references, including:

a) IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
b) Securities and Exchange Board of India (SEBI) Study on Analysis of Profit and Loss of Individual Traders in Equity Derivatives Segment (August 2026).
c) RFC 7519 – JSON Web Token (JWT) Specification.
d) FastAPI Official Documentation and Asynchronous Python Architecture Standards.
e) PostgreSQL Relational Database System Specification.

These references provide the structural, cryptographic, architectural, and documentation foundations upon which the system is designed.

2] Overall Description

2.1 Product Perspective

Project Vhagar is a web-based behavioral finance platform engineered to enhance investor decision-making discipline by capturing, analyzing, and mitigating cognitive biases during trading operations. It operates as an analytical intelligence middleware layer between user trading interfaces and execution services.

To address the high rate of retail trader losses highlighted in recent empirical research (SEBI 2026), Project Vhagar tracks user order flow, frequency, and risk parameters in real time. Upon detecting cognitive anomalies or panic trading patterns, the system triggers real-time alerts and temporary trade interruption protocols.

The system follows a three-tier architectural model consisting of:

a) Presentation Layer
b) Application Layer
c) Data Layer

The Presentation Layer is implemented in React.js, providing highly responsive user interfaces, real-time behavioral analytics widgets, and administrative configuration control panels.

The Application Layer is developed in Python using the FastAPI framework. This tier processes incoming trade telemetry, executes asynchronous bias detection algorithms, evaluates mathematical risk models, and enforces trade interruption rules via REST APIs.

The Data Layer relies on PostgreSQL to store user profiles, trade telemetry logs, cognitive bias evaluation parameters, and administrative audit trails under strict relational consistency.

The architecture prioritizes low latency and asynchronous event handling to ensure behavioral evaluations do not impede normal order execution paths during standard market conditions.

The system is intended for deployment within secure institutional environments such as police departments or forensic storage facilities. It complements investigative systems but does not replace case management platforms or forensic analysis tools.

2.2 Product Functions

Project Vhagar provides a suite of functional capabilities designed to evaluate and guide retail trader behavior.

The primary functional capabilities include:

a) Real-Time Telemetry and Order Pattern Logging
b) Cognitive Bias Detection and Scoring Engine
c) Trade Interruption and Cooling-Off Protocol Execution
d) Behavioral Analytics and Performance Dashboard
e) User Profile and Administrative System Management

Each function contributes to the overall objective of maintaining a secure and transparent chain of custody.

Real-time Telemetry and Order Pattern Logging captures incoming trade data points, execution timestamps, and position sizes to construct a behavioral baseline for each active user.

Cognitive Bias Detection algorithms evaluate deviation from baseline discipline, identifying overtrading, revenge trading, and emotional loss-chasing.

Trade Interruption Protocols automatically enforce temporary trading freezes when a user exceeds pre-configured risk parameters or demonstrates acute behavioral distress.

Behavioral Analytics Dashboards render actionable insights, psychological feedback, and risk metrics to help users mitigate disciplined execution failures.

User and Role Management controls access to system features using Role-Based Access Control policies. Different user roles have clearly defined permissions.

Reporting and Activity Logging provide structured documentation of custody history and system usage, supporting legal and administrative requirements.

2.3 User Classes and Characteristics

The system supports multiple user categories, each with distinct operational responsibilities and technical expectations.

The system defines two primary user classes:

a) Retail Traders
b) System Administrators

Retail Traders are individual market participants utilizing the platform to track their trading discipline, analyze cognitive metrics, and receive automated trade interruption protection during periods of market volatility or emotional bias.

System Administrators oversee system configuration, model threshold parameters, infrastructure performance, user management, and security policy enforcement.

Each user class operates under Role-Based Access Control restrictions to prevent unauthorized actions and maintain operational accountability.

2.4 Operating Environment

The Evidence-Locker system is designed to operate within a secure institutional IT infrastructure.

Client-Side Environment:
Modern web browser (Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge) with JavaScript enabled. The user interface is rendered via a React.js single-page application executing on standard desktop or mobile hardware.

Server-Side Environment:
Linux-based operating system (Ubuntu Server 22.04 LTS or higher), Python 3.10+ runtime, FastAPI microservices engine, and PostgreSQL 14+ relational database management system.

Network Environment:
The system is intended to operate within a secure Local Area Network or controlled cloud infrastructure. Firewall rules must restrict unauthorized external access. Secure communication between client and server must occur over encrypted HTTPS channels.

The system is not intended for deployment in unsecured public network environments.

2.5 Design and Implementation Constraints

The following constraints apply to system design and implementation:

a) The frontend web application must be developed using React.js.
b) Backend analytical services must be built using Python and FastAPI.
c) PostgreSQL must serve as the primary database management system.
d) Authentication must utilize JSON Web Tokens (JWT).
e) All client-server communication must be encrypted using TLS/HTTPS.
f) Latency for real-time telemetry processing must remain within strict sub-second parameters.

Legal and institutional policies may impose additional constraints regarding data retention and audit record preservation.

2.6 User Documentation

The system shall provide structured documentation to ensure proper usage and maintenance.

The documentation shall include:

a) Retail Trader User Guide detailing dashboard interpretation and interruption configuration.
b) Administrator System Operation Manual for system monitoring and bias model threshold tuning.
c) API Reference Manual for backend endpoint structures and integration specifications.

Documentation shall be written in clear procedural language and include workflow explanations.

2.7 Assumptions and Dependencies

The system assumes that users possess valid institutional credentials and are authorized to access evidence management systems. It assumes availability of secure hardware devices capable of running modern web browsers and supporting camera functionality.

The system depends on:

a) Python runtime and FastAPI library ecosystem.
b) PostgreSQL database service availability.
c) React.js frontend library dependencies.
d) Reliable network connection between client interfaces and analytical services.

If these dependencies fail or are misconfigured, system functionality may be affected.

3 External Interface Requirements

3.1 User Interfaces

The Project Vhagar platform provides a web-based client interface engineered to deliver real-time behavioral analytics, market sentiment insights, and execution controls to retail investors. The presentation layer is implemented as a single-page application using React.js and utilizes Plotly.js for rendering high-throughput multi-modal quantitative visualizations.

The platform user interface provides role-specific views configured dynamically according to authenticated permissions. The primary user interface components encompass:

a) Authentication and Session Management Interface
b) Multi-Modal Interactive Charting Module
c) Ripple Effect Domino Heatmap Interface
d) Behavioral Firewall Execution Control Modal
e) Devil's Advocate RAG Chatbot Interface
f) Administrator Model Threshold and Parameter Dashboard

Authentication and Session Management Interface:
The authentication view enforces OAuth2 standards with JSON Web Tokens (JWT). Upon successful authorization, user sessions maintain token state securely in client memory. All unauthorized route transitions shall be intercepted and redirected to the credential validation view with appropriate status indicators.

Multi-Modal Interactive Charting Module:
Renders synchronized time-series visualizations combining technical market telemetry from the Bombay Stock Exchange (BSE) with natural language processing (NLP) sentiment polarity metrics. Plotly.js capabilities enable sub-second pan, zoom, and overlay toggling across historical and streaming data horizons.

• Real-time candlestick data overlays with FinBERT sentiment scores.
• Interactive annotations depicting high-divergence anomaly events.
• Cross-asset sentiment correlation filters.
• Customizable technical indicators synced with news volume spikes.

The graphical user interface components adhere to responsive web standards, ensuring structural alignment and functional responsiveness across desktop workstations and mobile tablet viewports.

Ripple Effect Domino Heatmap Interface:
Provides an interactive topological matrix mapping supply chain interdependencies and cross-sectoral contagion risks. Sectoral nodes and asset edges adjust color intensity dynamically based on systemic risk transmission coefficients computed by backend machine learning microservices.

Behavioral Firewall Execution Control Modal:
Executes an active, non-bypassable overlay when the calculated Divergence Score exceeds configured risk thresholds. The modal displays real-time quantitative divergence metrics, details irrational herd behavior patterns, and initiates mandatory cooling-off countdown timers prior to order re-submission.

Devil's Advocate RAG Chatbot Interface:
Presents a conversational interface facilitating structured debiasing dialogues. The interface displays retrieve-augmented synthesis outputs, citing contradictory news sources and counter-thesis metrics alongside probability confidence scores to challenge user confirmation bias.

Administration Interface:
Restricted panel allowing authorized administrators to monitor model evaluation metrics, calibrate XGBoost/Random Forest feature weights, adjust FinBERT sentiment sensitivity factors, and inspect user activity audit logs.

3.2 Hardware Interfaces

Project Vhagar interacts with client host processing hardware and server infrastructure necessary to perform low-latency microservice executions and high-density graphical rendering.

Client-Side Hardware Acceleration:
The client workstation must incorporate a Graphics Processing Unit (GPU) supporting WebGL rendering pipelines to enable hardware-accelerated execution of Plotly.js chart computations and real-time DOM updates during multi-modal data stream rendering.

Server Compute Nodes:
Backend analytical microservices execute on multi-core compute clusters equipped with high-frequency CPUs (minimum 8 physical cores per instance) and dedicated Tensor Processing Units (TPUs) or NVIDIA Tensor Core GPUs to support real-time FinBERT embedding inferences and vector similarity calculations.

Network Compute Hardware:
Low-latency network interface cards (NICs) configured with hardware TCP offloading to maintain sub-100ms API round-trip transport times between web servers and upstream market feed handlers.

3.3 Software Interfaces

Project Vhagar interfaces with heterogeneous external APIs, databases, and machine learning software libraries to facilitate data ingestion, analytical processing, and client delivery.

Bombay Stock Exchange (BSE) Live Data Feed Interface:
Consumes streaming equity telemetry via secure WebSocket connections and RESTful polling endpoints. The ingestion service parses structured JSON payloads containing TICK, OHLCV (Open, High, Low, Close, Volume), and order book market depth metrics.

News API and Financial Media Ingestion Pipeline:
Interfaces with external financial news endpoints (e.g., Bloomberg API, Reuters API, and global News API services) via REST endpoints. The pipeline extracts unstructured textual news articles, press releases, and social sentiment strings for automated preprocessing.

FastAPI Machine Learning Framework Interface:
The Python-based FastAPI backend exposes asynchronous OpenAPI-compliant endpoints to client applications. It integrates PyTorch for FinBERT sentiment model execution, Scikit-Learn/XGBoost for machine learning classification, and LangChain/LlamaIndex for Retrieval-Augmented Generation workflows.

PostgreSQL and Vector Storage Interface:
The backend connects to PostgreSQL via AsyncPG connection pools to preserve transactional states, user telemetry, and audit logs. Unstructured news text embeddings are stored and queried using the pgvector extension for high-performance cosine similarity retrieval.

3.4 Communications Interfaces

Communication protocols enforce strict security, data integrity, and low-latency transport across all system nodes.

HTTPS / TLS 1.3 Transport:
All RESTful API transactions between the React.js client and FastAPI backend services shall be encrypted using Transport Layer Security (TLS 1.3) protocols over HTTPS.

WebSocket Protocol (WSS):
Bi-directional secure WebSocket connections maintain continuous streaming of market ticker updates, sentiment scores, and real-time firewall intervention signals.

Data Formatting Standard:
All request and response payloads, microservice inter-process communications, and event messages shall be formatted as strict JSON schema objects.

4 System Features

This section details the functional requirements of Project Vhagar: A Behavioral Finance Intelligence Platform. Each core feature is specified with rigorous requirements governing quantitative data processing, natural language evaluation, machine learning inference, and behavioral debiasing workflows. All requirements described below are mandatory.

4.1 Feature 1 – Multi-Modal Interactive Charting

4.1.1 Description
The Multi-Modal Interactive Charting feature ingests live quantitative market data from the Bombay Stock Exchange (BSE) and overlays natural language processing (NLP) sentiment polarity scores derived from real-time news sources. It provides synchronized graphical rendering of technical price action alongside market sentiment indicators, enabling users to evaluate emotional market overreactions.
The system executes automated text tokenization, sentiment extraction via FinBERT transformers, and time-series alignment using Plotly.js client rendering engine.

4.1.2 Functional Requirements
REQ-4.1.1
The system shall ingest live BSE tick data via WebSocket feeds with an ingestion latency not exceeding 200 milliseconds.
REQ-4.1.2
The system shall fetch financial news text strings continuously from registered News API endpoints at configured intervals of 60 seconds.
REQ-4.1.3
The backend microservice shall process ingested news articles using a fine-tuned FinBERT transformer model to extract sentiment polarity scores ranging from -1.0 (extremely negative) to +1.0 (extremely positive).

REQ-4.1.4
The system shall align calculated sentiment polarity scores with time-series OHLCV market data based on UTC timestamps.
REQ-4.1.5
The frontend client shall render multi-modal charts using Plotly.js, overlaying technical price candles with a continuous sentiment trend ribbon.
REQ-4.1.6
The charting interface shall allow users to toggle individual data layers, including volume profiles, moving averages, and sentiment confidence intervals.
REQ-4.1.7
The system shall automatically generate visual markers on the chart canvas when sentiment polarity diverges from price action direction by more than 2.5 standard deviations.
REQ-4.1.8
The interface shall support interactive hover cards displaying underlying news headlines, publisher metadata, and FinBERT confidence metrics for selected sentiment data points.
4.1.3 Post-Conditions
• Synchronized quantitative price and qualitative sentiment data stream continuously to client viewports.
• Sentiment-price divergence anomalies are highlighted for downstream behavioral analysis.

4.2 Feature 2 – Ripple Effect Domino Heatmap

4.2.1 Description
The Ripple Effect Domino Heatmap models and visualizes supply chain interdependencies, cross-asset correlations, and systemic contagion risks. By evaluating inter-industry linkages, the module forecasts cascading shocks across interconnected equities following primary market disruption events.
The system calculates directional transmission coefficients using graph neural networks and historical covariance matrices to quantify downstream vulnerability.

4.2.2 Functional Requirements
REQ-4.2.1
The system shall construct a directed asset correlation graph utilizing historical BSE price co-movements and sector supply-chain dependency trees.
REQ-4.2.2
The backend microservice shall compute cross-asset contagion coefficients upon receiving price or volume shock signals exceeding pre-set volatility thresholds.
REQ-4.2.3
The system shall dynamically generate a 2D matrix heatmap rendering nodes color-coded according to predicted risk spillover magnitude (green = low contagion risk, red = extreme systemic risk).
REQ-4.2.4
The heatmap module shall support interactive node selection, expanding downstream sub-sector dependencies and primary input supplier lists upon user click.
REQ-4.2.5
The system shall execute scenario simulation routines, allowing users to introduce synthetic price shocks to chosen anchor assets and rendering predicted multi-tier shock propagation paths within 500 milliseconds.
REQ-4.2.6
The system shall filter heatmap views based on user-selected risk parameters, including supply chain node depth, market capitalization tier, and sector classification.
4.2.3 Post-Conditions
• Sectoral domino impact graphs are updated and stored in cache.
• Systemic risk propagation alerts are routed to user dashboards.

4.3 Feature 3 – The Behavioral Firewall

4.3.1 Description
The Behavioral Firewall serves as an active trade execution interruption module engineered to prevent emotion-driven trade decisions, revenge trading, and herd mentality participation. It continuously evaluates user order telemetry against prevailing market sentiment, computing a real-time Divergence Score using Random Forest and XGBoost ensemble algorithms trained on FinBERT sentiment features.
When the computed score reveals anomalous behavioral divergence or irrational panic, the system intercepts trade dispatch calls, enforcing a mandatory cooling-off period.

4.3.2 Functional Requirements
REQ-4.3.1
The system shall intercept user order submissions prior to backend order router dispatch.
REQ-4.3.2
The backend microservice shall extract user order parameters, including trade frequency, position scaling factor, portfolio loss state, and order execution velocity.
REQ-4.3.3
The system shall calculate a Divergence Score (0.0 to 100.0) by feeding order parameters alongside FinBERT-derived market sentiment vectors into trained Random Forest and XGBoost classification models.
REQ-4.3.4
The system shall trigger an active trade pause if the calculated Divergence Score exceeds the user-configured risk threshold (default value = 75.0).
REQ-4.3.5
Upon firewall activation, the client interface shall lock order submission controls and display a mandatory cooling-off countdown modal (duration = 300 seconds).
REQ-4.3.6
The modal shall render quantitative diagnostic feedback explaining the detected bias factor (e.g., "Loss-Chasing Behavior Detected during Negative Market Sentiment Divergence").
REQ-4.3.7
The system shall mandate explicit user re-acknowledgement of strategy rules before releasing the order execution lock upon timer expiration.
REQ-4.3.8
All firewall intervention events, Divergence Scores, and user order attributes shall be logged securely in PostgreSQL audit tables for post-trade discipline review.

4.3.3 Integrity Guarantees
• Trade execution locks cannot be bypassed via browser client DOM manipulation.
• All Divergence Score decisions execute deterministically in backend microservices.

4.4 Feature 4 – Devil's Advocate Chatbot

4.4.1 Description
The Devil's Advocate Chatbot is a Retrieval-Augmented Generation (RAG) conversational agent engineered to challenge user confirmation bias. When a user formulates a bullish or bearish trade thesis, the chatbot retrieves counter-narrative financial news articles, opposing analyst ratings, and conflicting metric trends, engaging the user in a structured debiasing debate.
The module links natural language parsing with backend vector similarity engines to synthesize evidence-backed counter-arguments in real time.

4.4.2 Functional Requirements

REQ-4.4.1
The system shall capture user natural language investment thesis inputs via the conversational chat interface.
REQ-4.4.2
The NLP pipeline shall parse input text to identify target ticker symbols, directional bias (bullish/bearish), key rationale metrics, and time horizons.
REQ-4.4.3
The system shall convert user thesis statements into dense vector embeddings using the FinBERT embedding model.
REQ-4.4.4
The RAG engine shall query the PostgreSQL pgvector database to perform cosine similarity searches, retrieving top-k financial news fragments possessing inverse sentiment polarity scores.
REQ-4.4.5
The system shall feed retrieved counter-evidence texts along with the original user thesis into a Large Language Model (LLM) prompt context window.
REQ-4.4.6
The chatbot microservice shall generate structured conversational responses presenting opposing facts, citing news sources, and highlighting unconsidered tail-risk factors within 2.0 seconds.
REQ-4.4.7
The system shall compute a Confirmation Bias Score (0% to 100%) quantifying the degree to which the user ignores retrieved counter-evidence during subsequent order modifications.
REQ-4.4.8
The interface shall render hyperlinked source references for all retrieved counter-evidence citations utilized during the debate session.

4.4.3 Post-Conditions

• Counter-thesis arguments and supporting citations are presented to the user.
• User Confirmation Bias Score metrics are saved to the user profile database.

5. Non-Functional Requirements

This section specifies the non-functional quality attributes, operational performance bounds, security protocols, and architectural constraints governing Project Vhagar: A Behavioral Finance Intelligence Platform. All parameters are defined to ensure sub-second analytical execution, explainability, high concurrency, and data protection in live trading environments.

5.1 Performance and Latency Requirements

The platform shall maintain strict ultra-low latency processing limits to prevent trade execution degradation during volatile market regimes.
REQ-5.1.1
The real-time market data ingestion pipeline shall ingest and process live BSE tick feeds with an end-to-end transport and parsing latency not exceeding 50 milliseconds.
REQ-5.1.2
The Behavioral Firewall microservice shall compute the user Divergence Score and execute trade interception evaluations within a maximum processing window of 100 milliseconds from order dispatch initiation.
REQ-5.1.3
FinBERT sentiment transformer models shall complete text classification and polarity scoring on incoming news articles within 200 milliseconds per document payload.
REQ-5.1.4
The PostgreSQL data layer shall support a minimum throughput of 5,000 concurrent active database connections using AsyncPG pooling without exceeding a 10-millisecond query response latency.

REQ-5.1.5
The client React.js dashboard shall update Plotly.js chart overlays and risk heatmaps within 16 milliseconds (60 FPS rendering cycle) upon receiving WebSocket state payloads.

5.2 Security and Communications Requirements

System architecture shall enforce strict network communication security, identity verification, and data protection protocols across all tier interfaces.

5.2.1 Transport and Network Security

REQ-5.2.1
All HTTP REST client-server communications shall strictly enforce TLS 1.3 encryption over HTTPS. Legacy TLS protocols (<1.3) shall be explicitly rejected.
REQ-5.2.2
Real-time market and sentiment telemetry feeds delivered to web clients shall utilize secure WebSocket protocol (WSS) authenticated via short-lived JWT tokens.
REQ-5.2.3
Authentication endpoints shall issue OAuth2-compliant JSON Web Tokens (JWT) signed using HMAC SHA-256 or RSA-2048 encryption algorithms with mandatory 15-minute expiration windows.
REQ-5.2.4
User passwords and sensitive credentials stored in PostgreSQL shall be salted and hashed using Argon2id or bcrypt hashing algorithms.

5.2.2 Explainable AI (XAI) and Decision Transparency

REQ-5.2.5
All machine learning models evaluating user cognitive bias (including Random Forest and XGBoost classifiers) shall expose feature importance metrics for every calculated Divergence Score.
REQ-5.2.6
The system shall extract SHAP (SHapley Additive exPlanations) feature attribution values for each trade intervention event to quantify the relative weight of trade frequency, loss state, and market sentiment.
REQ-5.2.7
The Devil's Advocate RAG Chatbot shall consume model feature importance vectors and SHAP attribution values to generate dynamic natural language justifications explaining firewall activations to the user.
REQ-5.2.8
All model decision trees, hyperparameters, and feature weights shall be versioned and logged alongside bias evaluation outputs to ensure post-hoc auditability.

5.3 Reliability and System Availability Requirements

REQ-5.3.1
The analytical backend microservices and database layer shall maintain 99.9% operational availability during exchange trading hours.
REQ-5.3.2
Automated health-check mechanisms shall monitor FastAPI worker processes and initiate microservice failover within 2.0 seconds of process fault detection.
REQ-5.3.3
In the event of network disruption or market feed disconnection, the platform shall enter a fail-safe state, locking trade submission controls and alerting the user.

5.4 Scalability and Maintainability Requirements

REQ-5.4.1
The backend microservice layer shall be containerized using Docker and orchestratable via Kubernetes to support horizontal scaling during high-volatility volume surges.
REQ-5.4.2
Database schema updates and migrations shall be managed using Alembic migration tools under strict version control.

6. Appendices

This section contains supplementary domain glossaries, technical architectural specifications, database schema structures, and academic references for Project Vhagar.

6.1 Appendix A – Glossary

The following domain-specific terms, acronyms, and algorithmic definitions apply throughout this Software Requirements Specification:

Divergence Score

A quantitative metric (ranging from 0.0 to 100.0) generated by ensemble machine learning algorithms that measures the statistical deviation between user trading execution parameters and prevailing sentiment/technical market indicators.

FinBERT

A specialized Financial BERT (Bidirectional Encoder Representations from Transformers) language model fine-tuned for sentiment analysis and NLP processing of financial text corpora.

FOMO (Fear Of Missing Out)

A behavioral cognitive bias driving market participants to execute sub-optimal trades impulsively due to anxiety over missing prospective price rallies.

RAG (Retrieval-Augmented Generation)

An AI framework that enhances Large Language Model responses by retrieving relevant, authoritative facts from external vector databases prior to response synthesis.

Sentiment Polarity

A normalized continuous metric (ranging from -1.0 to +1.0) quantifying the qualitative emotional tone of financial media, where -1.0 represents extreme negativity and +1.0 represents extreme positivity.

XGBoost (Extreme Gradient Boosting)

An optimized, scalable distributed gradient-boosted decision tree algorithm utilized for high-performance classification and regression tasks.

6.2 Appendix B – System Models

6.2.1 Behavioral Firewall Execution Flow

The operational flow of the Behavioral Firewall follows a deterministic analytical sequence:

1. User submits order dispatch call via frontend client dashboard.
2. FastAPI middleware intercepts execution request before order router dispatch.
3. Analytical engine extracts user trade history, frequency, loss state, and live FinBERT market sentiment vectors.
4. Ensemble ML model (XGBoost/Random Forest) computes Divergence Score and SHAP feature importance metrics.
5. If Divergence Score exceeds threshold (>75.0), trade lock is enforced and 300-second cooling-off timer is triggered.

Chatbot renders natural language bias explanation using extracted feature attribution values.

6.3 Appendix C – Database Schema Overview

The relational PostgreSQL database structure for Project Vhagar comprises four primary core tables designed for high-concurrency transactional consistency:

1. Users Table

Stores trader identities, credential hashes, risk tolerance parameters, and authentication metadata.

• user_id (UUID, Primary Key)
• email (VARCHAR(255), Unique)
• password_hash (VARCHAR(255))
• risk_threshold_default (FLOAT, Default 75.0)
• created_at (TIMESTAMP WITH TIME ZONE)

2. TradeLogs Table

Logs all active and intercepted trade orders generated by registered users.

• trade_id (UUID, Primary Key)
• user_id (UUID, Foreign Key referencing Users)
• ticker_symbol (VARCHAR(20))

• order_type (VARCHAR(10)) -- BUY/SELL
• quantity (INTEGER)
• execution_price (NUMERIC(12,4))
• order_status (VARCHAR(20)) -- DISPATCHED/BLOCKED/COOLING_OFF
• timestamp (TIMESTAMP WITH TIME ZONE)
3. SentimentData Table
Stores financial news embeddings, FinBERT polarity scores, and publishing metadata.
• sentiment_id (UUID, Primary Key)
• news_article_id (VARCHAR(100))
• publisher (VARCHAR(100))
• headline (TEXT)
• polarity_score (FLOAT) -- Range -1.0 to +1.0
• embedding_vector (vector(768)) -- pgvector embedding
• published_at (TIMESTAMP WITH TIME ZONE)
4. BiasAudits Table
Preserves audit trails of behavioral evaluations, calculated Divergence Scores, and SHAP attributions.
• audit_id (UUID, Primary Key)
• trade_id (UUID, Foreign Key referencing TradeLogs)
• divergence_score (FLOAT)
• primary_bias_type (VARCHAR(50)) -- LOSS_CHASING/OVERTRADING/HERD_BEHAVIOR
• shap_attribution_json (JSONB) -- Explanatory feature importance weights
• firewall_activated (BOOLEAN)
• evaluated_at (TIMESTAMP WITH TIME ZONE)

6.4 Appendix D – Future Enhancements

The following architectural enhancements are scheduled for post-v1.0 releases of Project Vhagar:

1. Integration with direct exchange DMA (Direct Market Access) fix-protocol gateways.
2. Biometric EEG and eye-tracking hardware telemetry integration for physical stress detection.

Multi-broker account synchronization via standardized REST API aggregation middleware.

6.5 Appendix E – References

The technical specifications in this SRS derive from the following foundational references:

IEEE Std 830-1998
IEEE Recommended Practice for Software Requirements Specifications.
Securities and Exchange Board of India (SEBI)
Study on Analysis of Profit and Loss of Individual Traders in Equity Derivatives Segment (August 2026).
Araci, D. (2019)
FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. arXiv preprint arXiv:1908.10063.
Lundberg, S. M., & Lee, S. I. (2017)
A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems (NeurIPS).
RFC 7519
JSON Web Token (JWT) Standard.